插件化软件系统中的架构一致性检查

Architectural Consistency Checking in Plugin-Based Software Systems

Timo Greifenberg Klaus Müller Bernhard Rumpe
软件工程系，亚琛工业大学（RWTH Aachen University）
http://www.se-rwth.de

greifenberg@se-rwth.de mueller@se-rwth.de rumpe@se-rwth.de

摘要

手动确保软件系统的实现与软件架构保持一致，是一项费力且容易出错的任务。因此，人们开发了多种自动化一致性检查方法来对抗架构侵蚀。然而，这些方法缺乏针对插件依赖关系定义和检查架构约束的手段，而这正是插件化软件系统所必需的。

在本文中，我们提出了一种名为依赖约束语言（Dependency Constraint Language，DepCoL）的领域特定语言，用于方便地定义有关插件依赖的约束。使用 DepCoL，可以定义作用于插件组的约束，从而减少所需的规格说明工作量；可以仅针对特定插件制定约束；还可以对约束进行细化。

此外，我们提供了一个 Eclipse 插件，用于检查正在开发的软件系统是否与所建模的约束保持一致。这使一致性检查能够无缝集成到开发过程中，从而在软件系统开发期间轻松地检查一致性。通过这种方式，开发人员能够立即获知依赖违规情况，这有助于开发人员对抗架构侵蚀。

类别与主题分类：D.2 [Software Engineering]: Software Architectures

性、依赖约束语言

1. 引言

软件系统的软件架构定义了软件系统如何划分为组件，以及这些组件之间预期如何交互 [12, 7]。此外，软件架构还定义了对于理解软件系统至关重要的设计决策 [15]。然而，如果软件系统的架构被记录了下来，这些架构描述也很少得到维护 [9]。这是因为手动确保软件架构与软件系统的实现保持一致是一项费力且容易出错的任务。

这种架构与实现的偏离被称为架构侵蚀 [12]，它对软件质量、可维护性和可演化性都有负面影响 [9, 16, 8]。

为了应对这一问题，人们开发了多种方法来自动检查这种一致性 [9, 11, 2]。例如，这些方法提供了定义架构规则的手段，这些规则必须被实现所满足，或不得被实现所违反。另一些方法则基于对预期架构的定义和对现有架构的提取 [10, 3]。一旦检测到偏离，开发人员就会被告知。

然而，这些方法主要关注源代码实体（如类）之间（细粒度的）依赖关系，并且不允许方便地定义关于插件之间依赖的更为抽象的约束。用现有方法表达类似的约束在一定程度上是可能的，但对于需要为插件依赖定义约束的系统架构师来说，这更加费力且不够方便。特别地，本文假设被检查一致性的插件化软件系统是使用 Eclipse [6] 开发的，因此由 Eclipse 插件和 Eclipse 特性（feature）组成。尽管如此，只要能够识别插件之间以及插件组之间的依赖关系，我们的方法就可以移植到使用其他工具基础设施开发的插件化系统中。

本文提出了一种领域特定语言（DSL），称为依赖约束语言（DepCoL），用于在插件化软件系统中定义关于插件和特性依赖的约束。通过为插件化软件系统的架构师提供一种易于使用且方便的 DSL，我们假设该方法被接受并永久集成到开发过程中的可能性更高。使用 DepCoL，可以针对单个元素以及元素组定义约束，从而减少规格说明工作量。为了处理用 DepCoL 建模的约束，我们开发了一个 Eclipse 插件，它可以集成到插件化软件系统的开发环境中。该插件能够自动检查正在开发的系统的实现与所建模的约束之间的一致性。这些检查可以在软件系统开发的同时进行。如果开发人员引入了根据约束规格说明被禁止的依赖，这一违规会立即报告给开发人员，以便违规行为可以立即得到修复。

本文的结构如下：在第 2 节中，我们阐述工作的背景，介绍重要的术语和 DepCoL 的应用环境。之后，我们在第 3 节中概述我们的方法。第 4 节详细介绍 DepCoL。第 5 节描述相关工作，第 6 节对本文进行总结。

2. 背景

为了避免误解，首先介绍相关术语的定义。由于 DepCoL 是在与工业伙伴的合作项目中开发的，因此接下来简要描述这项工作的应用环境。

允许为个人或课堂使用制作本作品全部或部分内容的数字或硬拷贝，前提是不以营利或商业优势
ECSAW '15, September 07 - 11, 2015, Dubrovnik/Cavtat, Croatia
c 2015 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ISBN 978-1-4503-3393-1/15/09...$15.00 DOI: http://dx.doi.org/10.1145/2797433.2797493
[GMR15] T. Greifenberg, K. Müller, B. Rumpe: Architectural Consistency Checking in Plugin-Base

2.1 术语定义

本文关注的是使用 Eclipse 开发的插件化软件系统。因此，它涉及 Eclipse 插件与 Eclipse 特性之间的依赖关系。在本文的后续部分中，Eclipse 插件将简称为插件，Eclipse 特性简称为特性。

本工作中考虑的插件化系统的最小组成单元是插件。简单来说，插件是类以及这些类所需的库的集合。多个插件可以分组到一个特性中。在 Eclipse 中，特性还可以包含其他特性，但为了简化描述，下文假设特性仅包含插件。此外，本方法使用术语“特性组”来指代特性的逻辑分组。用户可以自由定义哪些特性应包含在这样一个特性组中。与此类似，插件组表示插件的逻辑分组。

基于这些定义，特性和插件组都由插件组成。两者之间的区别在于：特性是插件化软件系统本身的必要组成部分，而插件组仅在依赖模型中定义，用于对选定的插件进行分组。为了能够在依赖模型层面上区分这两种插件分组，本文使用了不同的术语。

每个插件为了正常工作可能依赖许多其他插件。这意味着只有当这些插件也可用时，该插件才能被执行。因此，插件与特性之间、特性与特性之间等存在（间接的）依赖关系。下面将具体化“依赖”这一术语。

如果插件 “m” 的 Eclipse 清单文件（manifest file）中将 “n” 列为必需的 bundle（required bundle），则称插件 “m” 依赖插件 “n”。本文中提到的所有其他依赖都基于这一依赖定义。在更详细地解释这一点之前，请注意：最终，一个插件、一个特性、一个插件组和一个特性组都可以表示为一组插件。对于特性和插件组来说，这一点是显而易见的。单个插件可以用一个只包含该插件的插件组来表示。对于特性组，可以通过收集该特性组中所有特性所包含的插件来推导出最终的插件组。

设 X 和 Y 表示插件、特性、插件组或特性组，设 PG 为返回所传参数对应的插件组的函数。那么，如果存在插件 m（m 属于 PG(X)）和插件 n（n 属于 PG(Y)），使得插件 “m” 依赖插件 “n”，则 X 依赖 Y。

2.2 应用背景

依赖约束语言 DepCoL 是在与工业伙伴的合作项目中开发的。该合作项目涉及复杂的 Eclipse 富客户端应用，这些应用总共由大约 800 个插件和 100 个特性组成，并共享若干特性和插件。

为了支持这些插件化软件系统的演化，合作项目的目标是开发一种 DSL，能够定义有关这些软件系统的特性和插件之间依赖关系的约束。一个特殊的要求是，应该能够在不同的粒度级别上定义约束。该语言不仅应能够表达针对元素组的限制，还应能够表达针对特定元素的特定限制。另一个要求是，开发人员应尽快获知违反已定义约束的情况。第 4 节介绍由此产生的依赖约束语言 DepCoL。

3. 概述

所提出方法的关键思想是：存在一个显式的依赖模型，其中包含关于插件化软件系统的插件和特性之间依赖关系的约束。软件系统实现与依赖模型之间的一致性可以自动检查。图 1 给出了通过所提出的方法建立的基本工作流程的概览。

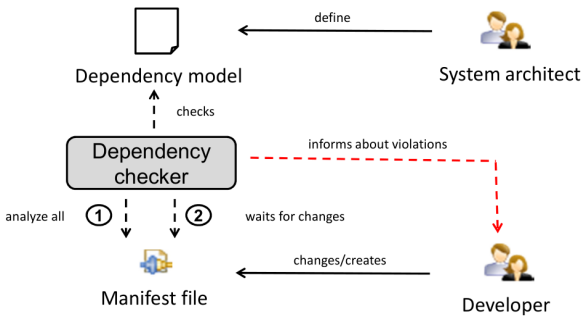


图 1：架构一致性检查过程概览

一开始，系统架构师必须使用 DepCoL 定义依赖模型。这个依赖模型封装了被分析的插件化软件系统不得违反的架构规则。

负责自动检查架构一致性的依赖检查器需要集成到开发环境中，即被检查的插件化软件系统的开发环境。然后，依赖检查器可以自动检查该软件系统相对于给定依赖模型的违规情况。为此，首先提取插件化软件系统的插件之间现有的依赖关系。由于本工作聚焦于使用 Eclipse 开发的插件化系统，现有依赖关系的提取通过分析 Eclipse 清单文件来完成。依赖检查器要么提取属于相应插件化软件系统的所有 Eclipse 清单文件中的全部依赖，要么被触发只提取某一个特定插件（以及关联的特性）的依赖。后者在依赖检查器被告知某个特定插件的依赖发生变化时执行。在这种情况下，依赖检查器只需要检查被修改的插件是否违反依赖模型，因此只需要获取这个被修改插件的依赖。

无论依赖检查器的调用以何种方式发起，开发人员都会通过如图 2 所示的 Eclipse 视图立即获知对依赖模型的违规情况。在该视图中，违规消息按违规的严重程度分组。如果检测到对某个特性的依赖被禁止或容忍，该视图还会指出该特性的哪些插件导致了违规。此外，违反依赖模型的插件的清单文件会被标注一个错误标记。这种即时反馈允许直接解决不期望的依赖，从而直接对抗架构侵蚀。

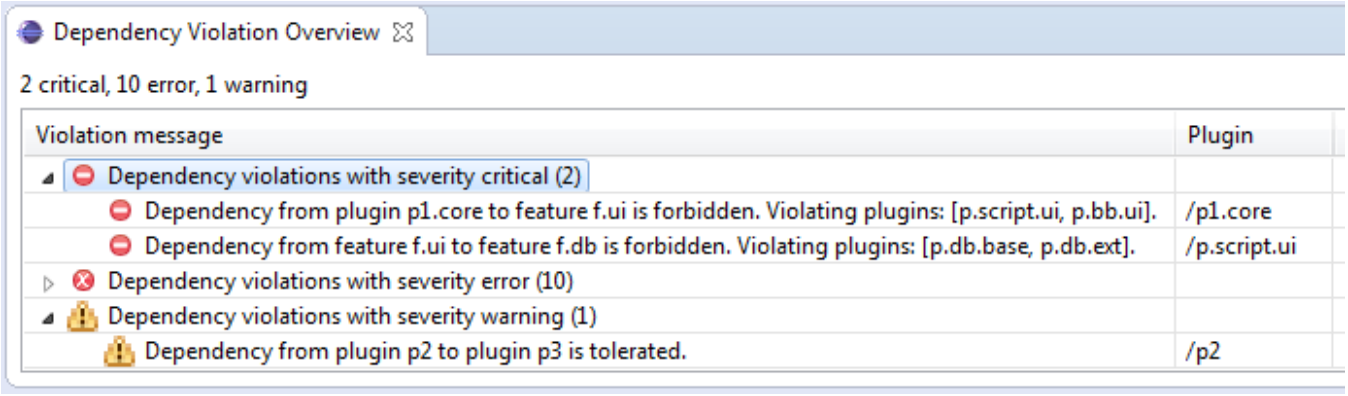


图 2：显示依赖检查器结果的 Eclipse 视图

4. 依赖约束语言 DepCoL

在本节中，介绍依赖约束语言 DepCoL。DepCoL 提供两类语言结构来定义依赖模型：一类允许定义引用插件或特性的结构，另一类使得能够制定关于插件、特性以及先前定义的结构之间依赖关系的约束。引用特性的结构的一个例子是特性组。在它的定义中，用户必须说明哪些特性属于该特性组。清单 2 显示了特性组定义的具体示例。一旦用户引入了诸如特性组之类的结构，就可以为结构中包含的元素声明具体的约束。例如，可以规定一个特性组中的所有特性不得依赖另一个特性组中的所有特性。

下面首先更详细地介绍 DepCoL 的不同语言结构。一个自包含但虚构的依赖模型示例可以在 [5] 中找到。之后，简要描述依赖模型的语义。最后，概述依赖检查器的实现。

```
declare featurebase {
f1;
f2;
f3;
}
```

清单 1：特性基示例

4.1 引用特性和插件的结构定义

4.1.1 特性基与插件基

依赖模型通常引用各种特性和插件名称。为了避免特性或插件名称在依赖模型中以不同的方式书写，引入了特性基（feature base）和插件基（plugin base）的概念。特性基包含可以在依赖模型中直接引用的所有特性的名称。一个上下文条件确保只有列在特性基中的特性名称才能在依赖模型中被命名。清单 1 显示了一个特性基规格说明的示例，通过它声明特性“f1”、“f2” 和 “f3” 可以在依赖模型中使用。

与此类似，所有可以被引用的插件名称都必须在一个插件基中声明。同样，这由一个上下文条件来确保。为了支持开发人员，所提供的 Eclipse 工具可以生成特性基和插件基，其中包含特定工作区中的所有特性和插件名称。

```
declare featuregroup fgListFeatures {
f1;
f2;
}
declare featuregroup fgRegExp {
fs.ui.*;
fs.core.*;
}
declare featuregroup fgCombined {
f3;
fs.ext.*;
featuregroup fgListFeatures;
featuregroup fgRegExp;
}
```

清单 2：特性组规格说明的不同变体

4.1.2 特性组与插件组

为了简化针对多个特性和插件的约束规格说明，DepCoL 支持特性组和插件组。特性组可以包含在特性基中定义的一系列特性名称。此外，特性组可以包含针对特性名称的正则表达式。在正则表达式中，可以使用通配符 “*”，它表示任意字符串。最后，特性组还可以包含其他特性组。总之，这意味着如果满足以下条件之一，则特性 “f” 包含在特性组 “fg” 中：

- 特性 “f” 被显式列在特性组 “fg” 中。
- 特性 “f” 匹配特性组 “fg” 中包含的一个正则表达式。
- 特性 “f” 包含在特性组 “subfg” 中，而 “subfg” 包含在 “fg” 中。

请注意，由于特性组中显式列出的每个特性名称都可以视为一个简单的正则表达式，并且特性组可以包含更多的正则表达式，因此每个特性组最终都可以表示为一个正则表达式，该正则表达式可能由多个备选组成，并表示特性的一个子集。与此类似，每个插件组也可以表示为一个正则表达式。

特性组规格说明的示例见清单 2。在该示例中，特性组 “fgListFeatures” 仅包含特性 “f1” 和 “f2”。特性组 “fgRegExp” 引用所有名称以

“fs.ui.” 或 “fs.core.” 开头的特性。最后，特性组 “fgCombined” 说明了如何组合不同的记法。该特性组包含先前描述的特性组中的特性，此外还包含特性 “f3” 以及所有名称以 “fs.ext.” 开头的特性。

与特性组规格说明类似，插件组可以包含多个显式声明的插件名称、针对插件的正则表达式以及其他插件组。特性组和插件组不必互斥，即一个特性可以包含在多个特性组中，一个插件也可以包含在多个插件组中。

除其他用途外，特性组和插件组可用于表示分层软件架构[1]。对于每一层，可以创建一个特性组或插件组，其中包含对应层中的特性或插件。此外，可以为每个特性组或插件组定义约束，以限制层之间的依赖关系。约束的定义将在下一小节中说明。

4.2 依赖约束的定义

可以为特性基中包含的所有特性以及插件基中列出的所有插件制定约束。此外，还可以为所有特性组和插件组声明约束。对于这些元素中的每一个，都可以定义：对特定特性、插件、特性组或插件组的依赖是被禁止、容忍还是允许的。默认情况下，所有元素之间的依赖都是允许的。因此，有必要显式地定义被禁止和容忍的依赖。尽管如此，有时也需要显式地允许某个依赖，这将在本小节的后面更详细地解释。如果需要，这种默认行为也可以调整。

容忍依赖可以用来表示：某个依赖不是期望的，但（暂时）也没有被禁止；或者暂时允许，但将来可能被禁止；或者依赖建模者不确定该依赖应该被允许还是被禁止。默认情况下，如果依赖检查器检测到定义为容忍的依赖，用户会收到一条警告消息。与此相反，如果检测到被禁止的依赖，则会报告一条错误消息。对于被禁止的依赖，可以标注一个严重级别，表示违反该规格说明的严重程度。该严重级别体现了修复违规行为的重要性。可能的严重级别类型有：严重（critical）、错误（error）和警告（warning）。在违规消息的报告中，消息根据其严重级别类型进行区分，如图 2 所示。

请注意，对于依赖检查器来说，严重级别为警告的被禁止依赖与容忍依赖被同等对待，因为两者都会产生警告消息。然而，对于依赖建模者来说，两者是有区别的。严重级别为警告的被禁止依赖确实是禁止的，但违规并不严重，报告一条警告就足够了。而容忍依赖通常表示该依赖不是期望的，需要明确如何处理该依赖。

```
plugin p1 {
[critical] forbid dependency to feature f2;
[warning] forbid dependency to featuregroup
fg2;
}
featuregroup fgListFeatures {
tolerate dependency to featuregroup fgRegExp;
}
```

清单 3：禁止/容忍依赖的示例

DepCoL 的一个重要特性是，约束可以细化（refine）先前为特定特性、插件、特性组和插件组定义的约束。通过这种方式，例如，可以表达插件组中的所有插件不得依赖特定插件，之后可以细化指出选定的插件被允许依赖特定的其他插件。细化概念将在接下来的小节中更详细地解释。

4.3 依赖模型的语义

如第 2.1 小节所解释的，插件、特性、插件组和特性组最终都可以表示为一组插件。设 X 和 Y 指代插件、特性、插件组或特性组，设函数 PG 返回所传参数所表示的插件组。那么，X 和 Y 之间的约束可以表示为插件对 (xi, yi) 的集合，其中所有 xi 属于 PG(X)，所有 yi 属于 PG(Y)。依赖模型中的每个约束都可以用这样的插件对来描述。

此外，每个插件对都被赋予一个属性，该属性指示该插件对所代表的依赖被视为严重、错误、警告还是允许。默认情况下，所有依赖都是允许的。因此，初始时所有可能的插件对都被赋予“允许”属性。对于每个约束，该属性根据约束类型而改变。例如，对于严重级别为错误的被禁止依赖，相应的插件对被赋予“错误”属性。

设 c1 和 c2 是依赖模型中的两个约束，设 PP 为返回每个约束的插件对的函数。当且仅当 PP(c1) 与 PP(c2) 的交集非空，且 c1 定义在 c2 之前时，约束 c2 细化约束 c1。PP(c1) 与 PP(c2) 的交集恰好包含被细化的插件对。如果一个约束细化了另一个约束，则由细化约束决定被细化插件对关联的属性。例如，如果约束 c1 描述了插件对之间依赖被视为错误的插件对，而约束 c2 描述了插件对之间依赖被视为严重的插件对，并且 c2 细化了
c1，那么所有被细化的插件对都被赋予“严重”属性。

总而言之，依赖模型描述了从插件对到属性的映射，该属性指示插件之间的依赖被视为允许、严重、错误还是警告。

4.4 依赖约束检查器的实现

在本节中，概述依赖检查器的实现，并通过一个具体示例加以说明。假设要检查某个特定插件的依赖以及包含该插件的特性的依赖是否违反给定的依赖模型。对所有插件的依赖进行验证的情况，通过对每个插件执行这些步骤来处理。

4.4.1 实现

作为依赖检查阶段的第一步，依赖检查器提取被分析软件系统中插件之间现有的依赖关系。只有在插件依赖之前未被提取过，或者插件依赖自上次提取以来发生了变化时，才会执行此提取。

在依赖检查器中必须考虑的最重要的方面是：约束可以被在依赖模型中稍后定义的约束所细化。为了处理这一点，依赖模型从后向前处理，定义约束的每个结构也从后向前遍历。在这种依赖模型遍历中，只有当某个依赖尚未被先前处理过的约束处理时，才会考虑该依赖，这将在后面更详细地解释。此外，只处理与被验证插件或包含被验证插

件的特性相关的约束。

在处理相关约束时，依赖检查器为插件对建立关系，根据依赖模型，这些插件对之间的依赖被视为允许，或被视为严重、错误或警告。这四个关系在下文中分别命名为 Rallowed、Rcritical、Rerror 和 Rwarning，它们实现了从为每个约束创建的插件对到依赖类型属性的映射，如上一小节所述。为了简化描述，下文中关系 R 指代所有四种关系变体的并集。

实现的一个特殊性是，对于每个约束，只考虑 (xi, yi) 对，其中 yi 是 xi 所需的插件。因此，R 只包含与被分析插件所需插件相关的信息。实现的另一个重要方面是，R 只存储为显式定义的约束而创建的插件对。如果没有关于两个插件 xi 和 yi 之间依赖的约束，则对 (xi, yi) 不包含在 R 中，并且被隐式地视为允许。

由于依赖模型是从后向前遍历的，并且约束可以细化其他约束，只有当 R 中尚未包含对 (xi, yi) 时，该对才会被添加到 R 中。否则，说明先前已经处理过某个约束，该约束细化了当前处理的约束。因此，所有 R 关系在构造上是不相交的。

最后，依赖检查器遍历关系 Rcritical、Rerror 和 Rwarning，并为其中包含的条目创建违规消息。

除了管理 R 中的插件对外，还存储了哪个具体约束导致了相应插件对的添加。一个用例是，这有助于创建违规消息。关系 Rwarning 包含插件对，这些插件对之间的依赖被视为警告。然而，在这个层面上，无法区分这些对是由于容忍依赖还是严重级别为警告的被禁止依赖而创建的。通过管理导致对添加的约束，可以区分这两种情况。因此，生成的违规消息可以指明该依赖是被禁止还是被容忍。此外，它有助于构造更好的违规消息。例如，如果依赖模型包含一个约束，表达某个特性被禁止依赖另一个特性，则可以创建一条准确表达这一点的违规消息，而不是仅仅报告两个插件之间的依赖被禁止。因此，生成的违规消息反映了依赖模型中的规格说明。

管理哪个约束导致哪个插件对添加的另一个用例是，这允许告知用户哪个约束细化了哪个其他约束。每次当对 (pi, pj) 应被添加到 R，但 R 已经包含该对时，可以记录下映射到该存储对的约束已经细化了当前处理的约束。通过将此日志报告给用户，依赖建模者可以验证细化是否有意为之。这样做的动机是，特别是在大型依赖模型中，依赖建模者可能并不总是意识到某个约束细化了先前的约束。

4.4.2 示例

下面概述清单 4 中给出的依赖模型的处理过程。在该示例中，假设插件基（下文用 PB 表示）包含插件“p1”、“p2”、“p3”、“p4.ui”、“p5.ui” 和“p6.i18n”。此外，假设插件组 “pg1” 至少包含插件“p1”，插件组 “pgUi” 包含所有名称以 “.ui” 结尾的插件，插件组 “ALL” 包含所有插件。最后，假设被验证的插件是 “p1”，且“p1” 需要插件 “p4.ui”、“p5.ui” 和“p6.i18n”。

由于被验证的插件是“p1”，因此只处理针对该插件的约束。根据清单 4 中第 6 行的约束，对 (p1, p4.ui) 被添加到 Rallowed。下一步，将处理插件组 “pg1” 的约束，从第 3 行的约束开始。基于该约束，将检查对 (p1, p4.ui) 和 (p1, p5.ui) 是否可以添加到 Rwarning。由于 Rallowed 已经包含 (p1, p4.ui)，因此只有 (p1, p5.ui) 被添加到 Rwarning。最后，对 (p1, px)（其中 px 属于 PB）被添加到 Rerror，前提是 px 表示一个被需要的插件并且 R 尚未包含相应的对。因此，只有 (p1, p6.i18n) 被添加。结果，将为从 “p1” 到 “p5.ui” 的容忍依赖报告一条警告消息，并为 “p1” 与“p6.i18n” 之间的依赖报告一条错误消息。

```
plugingroup pg1 {
forbid dependency to plugingroup ALL;
tolerate dependency to plugingroup pgUi;
}
plugin p1 {
allow dependency to plugin p4.ui;
}
```

清单 4：插件细化插件组约束的示例

5. 相关工作

在本节中，我们讨论静态架构一致性检查背景下的相关工作。我们的工作与现有方法之间最重要的区别在于，我们的工作聚焦于插件化软件系统。相比之下，大多数现有方法处理的是源代码层面上的依赖。

在 [14, 15] 中，使用静态的声明式语言 DCL 来限制软件系统内的依赖。DCL 提供了在模块之间定义约束的手段，模块表示类的集合。为了指定哪些类属于某个模块，可以通过标注类的完全限定名来列出这些类，或者使用可以匹配多个类的正则表达式。基于这些模型，可以定义若干约束，例如，只有模块 A 中的类才能访问模块 B 中的类。[14, 15] 中模块的定义与特性和插件组的定义类似。

在 [10] 中，介绍了软件反射模型（software reflexion model）技术。在该技术中，开发人员必须定义一个高层模型，其中包含实体以及这些实体之间的关系。此外，开发人员必须从源代码中提取一个源模型（例如调用图），最后必须在高层模型和源模型之间执行映射。在映射阶段，高层模型中的实体必须被分配给源模型的元素，通常使用正则表达式。基于这些模型和映射，计算软件反射模型，以确定高层模型在哪些地方（不）符合源模型。与反射模型相比，在我们的工作中，开发人员不必提供高层模型以及到源模型的映射。相反，定义的是表达允许、容忍或禁止依赖的约束。此外，很容易细化其他约束。

在 [13] 中，通过静态代码分析从源代码中提取依赖关系。得到的依赖关系显示在依赖结构矩阵（DSM）中。为了定义 DSM 中哪些类型的依赖被允许或被禁止，可以定义设计规则。这些规则应用于 DSM，以识别哪些依赖违反了

预期的软件架构。默认情况下，设计规则是继承的，即如果为某个子系统定义了限制，该子系统的所有元素都继承这些限制。此外，可以为 DSM 的特定元素细化设计规则。我们工作中细化约束的想法正是受到设计规则可以细化这一特性的启发。此外，约束也是继承的，例如，如果为特性组定义了约束，默认情况下，所包含特性的所有插件都继承这些约束。然而，开发人员也可以在 DCL 中表达容忍依赖。

应对架构侵蚀的另一种策略是使用查询语言 [18]。可以在源代码或其他相关工件上执行查询，例如，识别所有依赖特定其他类或包的类 [17]。因此，为了评估架构一致性，需要编写识别被禁止依赖的查询。

例如，[4] 将基于规则的方法应用于架构合规性检查。该方法基于形式化记号来表示预期的软件架构、系统的设计或实现，以及两者之间的映射。每个部分都被表示为一个逻辑知识库，其中包含表示描述元素的事实。通过在知识库的并集上执行架构规则来检查架构合规性。特别是，约束规则不仅可以报告对规则的违反，还可以识别违反的元素。在 [8] 中，这一思想得到了进一步的发展。架构规则被定义为在一个公共的可扩展本体上的逻辑公式，模型被映射到该本体的实例。为了以这种形式表示模型，必须定义转换，说明模型元素如何被表示。这使得可以使用模型来定义架构规则。因此，定义转换的开发人员主要需要逻辑编程方面的知识。与这些工作相比，我们的工作在任何层面上都不需要逻辑编程知识，但另一方面灵活性较低。然而，在我们的用例中并不需要这种灵活性，因为我们有意聚焦于插件化系统。通过这种方式，我们可以聚焦于熟悉 Eclipse 的开发人员和架构师所了解的方面。

关于静态架构合规性检查的更多方法和工具在 [11, 9] 中介绍。关于架构侵蚀以及最小化、预防和修复架构侵蚀的方法的详细综述见 [2]。

由于插件之间的依赖通常源于插件的类以某种方式相互依赖，因此使用我们的依赖检查器实现的一个替代方案是，将依赖模型转换为适合基于类的方法的模型。这样做的好处是，生成的模型可以用现有的基于类的工具进行检查。这种场景中的一个问题是，Eclipse 清单文件可能包含未使用的依赖，这些依赖无法追溯到类依赖。因此，基于类的方法无法检测这些依赖。为了避免这个问题，必须确保任何清单文件都不包含未使用的依赖。另一个潜在的问题是，多个插件可以包含同名的类——尽管这会是糟糕的实践。基于类的方法无法区分这些类。此外，每次清单文件更改后，都需要将依赖模型转换为基于类的模型。这些原因以及支持容忍依赖、约束细化和集成到 Eclipse 中的要求，最终促使我们决定实现我们自己的、针对我们的 DSL 定制的依赖检查器。

6. 结论

在本文中，我们提出了 DSL DepCoL，用于在插件化软件系统中定义关于插件和特性依赖的约束。使用 DepCoL，可以在不同的粒度级别上指定约束，例如针对特性组和插件组，也可以针对特定的特性和插件。DepCoL 的一个特色是它支持约束的细化。这有助于先定义一些通用约束，然后在需要时针对特定元素进行细化。

通过将依赖检查器集成到实现插件化软件系统的开发环境中，开发人员可以在其进行的更改违反先前定义的约束时立即获知。通过这种方式，DepCoL 及相应的工具在插件化软件系统中对抗架构侵蚀。该方法主要适用于具有大量插件的复杂插件化软件系统。在这类系统中，手动跟踪插件和特性之间的依赖关系是不可能的。

对于未来的工作，我们计划在工业合作项目中进行一项长期案例研究。在此过程中，我们特别计划调查我们目前使用的细化解释是否适用于复杂的插件化软件系统，或者是否应该使用其他信息，例如元素的层次结构。此外，我们计划分析考虑目前尚未考虑的传递依赖是否会是有利的。

7. 参考文献

[1] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal. Pattern-Oriented Software Architecture: A System of Patterns. John Wiley & Sons, Inc., New York, NY, USA, 1996.

[2] L. de Silva and D. Balasubramaniam. Controlling software architecture erosion: A survey. Journal of Systems and Software, 85(1):132-151, Jan. 2012.

[3] F. Deissenboeck, L. Heinemann, B. Hummel, and E. Juergens. Flexible Architecture Conformance Assessment with ConQAT. In Proceedings of the International Conference on Software Engineering, ICSE '10, pages 247-250, 2010.

[4] C. Deiters, P. Dohrmann, S. Herold, and A. Rausch. Rule-Based Architectural Compliance Checks for Enterprise Architecture Management. In Proceedings of the International Enterprise Distributed Object Computing Conference, EDOC '09, pages 183-192, 2009.

[5] Dependency model example. http://se-rwth.de/materials/saerocon/depModelExample.txt. Last visited on 24/06/2015.

[6] Eclipse website. http://www.eclipse.org/. Last visited on 24/06/2015.

[7] D. Garlan and M. Shaw. An Introduction to Software Architecture. Technical Report CMU-CS-94-166, Carnegie Mellon University, Pittsburgh, PA, USA, 1994.

[8] S. Herold and A. Rausch. Complementing Model-Driven Development for the Detection of Software Architecture Erosion. In Proceedings of the International Workshop on Modeling in Software Engineering, MiSE '13, pages 24-30, 2013.

[9] J. Knodel and D. Popescu. A Comparison of Static Architecture Compliance Checking Approaches. In Proceedings of the Working IEEE/IFIP Conference on Software Architecture, WICSA '07, pages 12-12, 2007.

[10] G. C. Murphy, D. Notkin, and K. J. Sullivan. Software Reflexion Models: Bridging the Gap between Design and Implementation. IEEE Transactions on Software Engineering, 27(4):364-380, Apr. 2001.

[11] L. Passos, R. Terra, M. T. Valente, R. Diniz, and N. Mendonca. Static Architecture Conformance Checking: An Illustrative Overview. IEEE Software, 27(5):82-89, Sept 2010.

[12] D. E. Perry and A. L. Wolf. Foundations for the Study of Software Architecture. ACM SIGSOFT Software Engineering Notes, 17(4):40-52, Oct. 1992.

[13] N. Sangal, E. Jordan, V. Sinha, and D. Jackson. Using Dependency Models to Manage Complex Software Architecture. In Proceedings of the Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA ’05, pages 167-176, 2005.

[14] R. Terra and M. T. Oliveira Valente. Towards a Dependency Constraint Language to Manage Software Architectures. In Proceedings of the European Conference on Software Architecture, ECSA ’08, pages 256-263, 2008.

[15] R. Terra and M. T. Valente. A dependency constraint language to manage object-oriented software architectures. Software - Practice & Experience, 39(12):1073-1094, Aug. 2009.

[16] R. Terra, M. T. Valente, K. Czarnecki, and R. S. Bigonha. Recommending Refactorings to Reverse Software Architecture Erosion. In Proceedings of the European Conference on Software Maintenance and Reengineering, CSMR ’12, pages 335-340, 2012.

[17] J. Van Eyck, N. Boucké, A. Helleboogh, and T. Holvoet. Using Code Analysis Tools for Architectural Conformance Checking. In Proceedings of the International Workshop on SHAring and Reusing Architectural Knowledge, SHARK ’11, pages 53-54, 2011.

[18] M. Verbaere, M. Godfrey, and T. Girba. Query Technologies and Applications for Program Comprehension. In International Conference on Program Comprehension, ICPC ’08, pages 285-288, 2008.